

## Assignment 2 Understanding transfer learning and fine tuning

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torch.nn.functional as F
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms
from torch.utils.data import random_split
from torchvision import datasets
import os
import numpy as np
from typing import Literal
from torchvision.models import resnet18, ResNet18_Weights
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from pprint import pprint

print(f"[INFO] Torch infos: {torch.__version__}")
```

[INFO] Torch infos: 2.6.0+cu124

## We need transform from resnet18 and its weights

```
In [2]: transform = ResNet18_Weights.DEFAULT.transforms()
print(transform)
```

```
ImageClassification(
  crop_size=[224]
  resize_size=[256]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BILINEAR
)
```

## pick a dataset you can import from

```
from torchvision import datasets
```

## Easy picks:

- Food101
- Flowers102
- DTD
- FGVAircraft

## Other picks:

```
full_train = datasets.OxfordIIITPet(root="data",
split="trainval", download=True, transform=transform)
test_ds     = datasets.OxfordIIITPet(root="data", split="test",
download=True, transform=transform)
```

## Split the data

```
In [3]: data = datasets.Food101(root="Dataset",split="train", transform=transform)
test_ds = datasets.Food101(root="Dataset",split="test", transform=transform)
```

## load the train, validation and test split

```
In [4]: print(data)
print(test_ds)
images= len(data)
# print(images)
val_ratio = 0.2
train_images = int(images *(1-val_ratio))
val_images = int(images*val_ratio)
class_names = data.classes
print (f'toatl images {images}, val images {val_images}, train images {train_images}')
pprint(len(class_names))
```

```
Dataset Food101
  Number of datapoints: 75750
  Root location: Dataset
  split=train
  StandardTransform
Transform: ImageClassification(
  crop_size=[224]
  resize_size=[256]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BILINEAR
)
Dataset Food101
  Number of datapoints: 25250
  Root location: Dataset
  split=test
  StandardTransform
Transform: ImageClassification(
  crop_size=[224]
  resize_size=[256]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BILINEAR
)
toatl images 75750, val images 15150, train images 60600
101
```

```
In [5]: train_split, val_split = random_split(data, [train_images, val_images])
```

## create dataloader handler from the dataset

```
In [6]: train_loader = DataLoader(train_split, batch_size=128, shuffle = True, num_
val_loader = DataLoader(val_split, batch_size=128, shuffle = False, num_wor
test_loader = DataLoader(test_ds, batch_size=128, shuffle = True, num_work
```

## now create the cnn, use the resnet18 as backbone, and ResNet18\_Weights as initial weights.

- create the backbone
- create a classifier
- based on your dataset add the correct number of output classes
- the classifier have to be trainable.

## Question 1?

- Why we freeze the backbone? Why not the classifier?

Because I am not training the model. I am using the already trained model so the model already knows how to recognise edges, textures and shapes. If i didnt freeze the backbone, that would trigger back propagation to re adjust the weights. I am freezing the backbone so the classifier can first learn how to interpret existing features specifically for Food101 labels.

## add the train evaluation, and predict functions here

```
In [7]: model = resnet18(weights=ResNet18_Weights.DEFAULT)
print(model.layer4)
print(model.fc)
```

```

Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=
(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_ru
nning_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_ru
nning_stats=True)
    (downsample): Sequential(
      (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_ru
nning_stats=True)
    )
  )
  (1): BasicBlock(
    (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
    (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_ru
nning_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=
(1, 1), bias=False)
    (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_ru
nning_stats=True)
  )
)
Linear(in_features=512, out_features=1000, bias=True)

```

```
In [8]: model.fc = nn.Linear(512, 101)
        pprint(model.fc)
```

```
Linear(in_features=512, out_features=101, bias=True)
```

I just realized that I dont need to do this manually and follow the function given in the slides. I hope that isnt an issue.

```
In [9]: #this was the class and method shown in the lecture.
class ResNet18Transfer(nn.Module):
    def __init__(self,
                  number_of_output_classes: int,
                  freeze_backbone: bool,
                  dense_units: list[int] = [256],
                  dropout_probabilities: float = 0.3,
                  weights=ResNet18_Weights.DEFAULT):

        super().__init__()

        self.weights = weights

        # create a feature extractor
        backbone = resnet18(weights=weights)
        self.features = nn.Sequential(*list(backbone.children())[:-1])
        self.backbone_out = backbone.fc.in_features

        # freeze the layers
        if freeze_backbone:
            for p in self.features.parameters():
```

```

        p.requires_grad = False

    # classifier
    mlp = []
    cur = self.backbone_out
    for h in dense_units:
        mlp += [
            nn.Linear(cur, h),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout_probabilities),
        ]
        cur = h

    # add final classifier
    self.classifier = nn.Sequential(*mlp)

    # create final layer
    self.final_classifier = nn.Linear(cur, number_of_output_classes)

    # ensure head trainable
    for p in self.classifier.parameters():
        p.requires_grad = True
    for p in self.final_classifier.parameters():
        p.requires_grad = True

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return self.final_classifier(x)

    def get_transform(self):
        # This returns the recommended preprocessing for the pretrained w
        return self.weights.transforms()

    def fine_tune(self, unfreeze=True):
        # This targets the last block of the feature extractor (layer4)
        for param in self.features[7].parameters():
            param.requires_grad = unfreeze

```

```

In [10]: def train_one_epoch(model, train_loader, optimizer, criterion, device="cu
model.train()
running_loss = 0.0
correct = 0
total = 0
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    logits = model(images)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()
    running_loss += loss.item() * images.size(0)
    preds = logits.argmax(dim=1)
    correct += (preds == labels).sum().item()
    total += labels.size(0)
return running_loss / len(train_loader.dataset), correct / total

```

```
In [11]: @torch.no_grad()
def evaluate(model, val_loader, criterion, device="cuda"):
    model.eval()
    correct, total = 0, 0
    running_loss = 0.0

    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        loss = criterion(logits, labels)
        current_image_loss = loss.item()
        running_loss += current_image_loss * images.size(0)

        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return running_loss/total, correct / total
```

```
In [12]: @torch.no_grad()
def predict(model, images, device, class_names, topk=1):
    model.eval()
    images=images.to(device)
    logits = model(images)
    probs = F.softmax(logits, dim=1)
    topk_probs, topk_indices = probs.topk(topk, dim=1)
    results = []
    for i in range(images.size(0)):
        preds = []
        for j in range(topk):
            idx = int(topk_indices[i, j].item())
            p = topk_probs[i, j].item()
            label = class_names[idx] if class_names else idx
            preds.append({"Label":label, "index":idx, "probability":p})
        results.append({"topk":preds})
    return results
```

## Phase 1: Transfer learning

# freeze all layers except the classifier.

- train and evaluate the model for 50 epochs
- remember to save val loss and train loss

```
In [13]: device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
```

Using device: cuda

```
In [14]: epochs = 50
best_val_acc = 0.0

train_loss_history = []
val_loss_history = []
train_acc_history = []
val_acc_history = []
```

```
model_params = {
    "number_of_output_classes":101,
    "freeze_backbone": True,
    "dense_units":[256],
    "dropout_probabilities":0.3
}
model = ResNet18Transfer(**model_params)

trainable_params = []
for p in model.parameters():
    if p.requires_grad:
        trainable_params.append(p)

optimizer = optim.Adam(trainable_params, lr=1e-3)

loss_fn = nn.CrossEntropyLoss()

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
for i in range(epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, loss_fn, device=device)
    train_loss_history.append(train_loss)
    val_loss_history.append(val_loss)
    train_acc_history.append(train_acc)
    val_acc_history.append(val_acc)
    print(f"Epoch {i+1}/{epochs} - Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_model.pth")
```

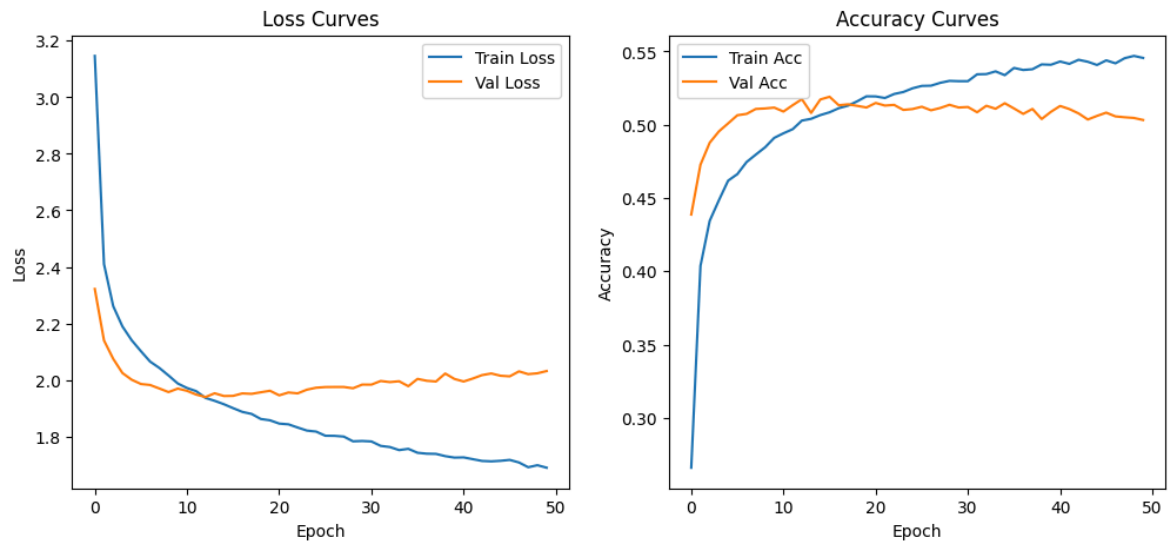
Epoch 1/50 - Train Loss: 3.1464, Train Acc: 0.2660, Val Loss: 2.3218, Val Acc: 0.4387  
Epoch 2/50 - Train Loss: 2.4098, Train Acc: 0.4037, Val Loss: 2.1395, Val Acc: 0.4727  
Epoch 3/50 - Train Loss: 2.2609, Train Acc: 0.4344, Val Loss: 2.0744, Val Acc: 0.4877  
Epoch 4/50 - Train Loss: 2.1890, Train Acc: 0.4485, Val Loss: 2.0242, Val Acc: 0.4954  
Epoch 5/50 - Train Loss: 2.1404, Train Acc: 0.4618, Val Loss: 2.0006, Val Acc: 0.5009  
Epoch 6/50 - Train Loss: 2.1016, Train Acc: 0.4663, Val Loss: 1.9855, Val Acc: 0.5065  
Epoch 7/50 - Train Loss: 2.0647, Train Acc: 0.4745, Val Loss: 1.9820, Val Acc: 0.5074  
Epoch 8/50 - Train Loss: 2.0424, Train Acc: 0.4796, Val Loss: 1.9695, Val Acc: 0.5108  
Epoch 9/50 - Train Loss: 2.0155, Train Acc: 0.4846, Val Loss: 1.9569, Val Acc: 0.5111  
Epoch 10/50 - Train Loss: 1.9873, Train Acc: 0.4910, Val Loss: 1.9692, Val Acc: 0.5117  
Epoch 11/50 - Train Loss: 1.9713, Train Acc: 0.4942, Val Loss: 1.9613, Val Acc: 0.5090  
Epoch 12/50 - Train Loss: 1.9601, Train Acc: 0.4969, Val Loss: 1.9480, Val Acc: 0.5133  
Epoch 13/50 - Train Loss: 1.9363, Train Acc: 0.5028, Val Loss: 1.9393, Val Acc: 0.5174  
Epoch 14/50 - Train Loss: 1.9260, Train Acc: 0.5040, Val Loss: 1.9526, Val Acc: 0.5081  
Epoch 15/50 - Train Loss: 1.9139, Train Acc: 0.5066, Val Loss: 1.9429, Val Acc: 0.5172  
Epoch 16/50 - Train Loss: 1.9000, Train Acc: 0.5085, Val Loss: 1.9434, Val Acc: 0.5191  
Epoch 17/50 - Train Loss: 1.8868, Train Acc: 0.5113, Val Loss: 1.9522, Val Acc: 0.5133  
Epoch 18/50 - Train Loss: 1.8796, Train Acc: 0.5130, Val Loss: 1.9506, Val Acc: 0.5139  
Epoch 19/50 - Train Loss: 1.8617, Train Acc: 0.5160, Val Loss: 1.9561, Val Acc: 0.5129  
Epoch 20/50 - Train Loss: 1.8570, Train Acc: 0.5194, Val Loss: 1.9615, Val Acc: 0.5117  
Epoch 21/50 - Train Loss: 1.8454, Train Acc: 0.5193, Val Loss: 1.9454, Val Acc: 0.5149  
Epoch 22/50 - Train Loss: 1.8428, Train Acc: 0.5182, Val Loss: 1.9554, Val Acc: 0.5131  
Epoch 23/50 - Train Loss: 1.8315, Train Acc: 0.5210, Val Loss: 1.9524, Val Acc: 0.5135  
Epoch 24/50 - Train Loss: 1.8206, Train Acc: 0.5223, Val Loss: 1.9653, Val Acc: 0.5101  
Epoch 25/50 - Train Loss: 1.8171, Train Acc: 0.5249, Val Loss: 1.9723, Val Acc: 0.5107  
Epoch 26/50 - Train Loss: 1.8024, Train Acc: 0.5265, Val Loss: 1.9747, Val Acc: 0.5123  
Epoch 27/50 - Train Loss: 1.8020, Train Acc: 0.5266, Val Loss: 1.9749, Val Acc: 0.5098  
Epoch 28/50 - Train Loss: 1.7994, Train Acc: 0.5286, Val Loss: 1.9750, Val Acc: 0.5114  
Epoch 29/50 - Train Loss: 1.7827, Train Acc: 0.5299, Val Loss: 1.9704, Val Acc: 0.5136  
Epoch 30/50 - Train Loss: 1.7838, Train Acc: 0.5297, Val Loss: 1.9834, Val Acc: 0.5117

Epoch 31/50 - Train Loss: 1.7822, Train Acc: 0.5297, Val Loss: 1.9832, Val Acc: 0.5121  
Epoch 32/50 - Train Loss: 1.7665, Train Acc: 0.5344, Val Loss: 1.9961, Val Acc: 0.5085  
Epoch 33/50 - Train Loss: 1.7624, Train Acc: 0.5345, Val Loss: 1.9922, Val Acc: 0.5129  
Epoch 34/50 - Train Loss: 1.7515, Train Acc: 0.5364, Val Loss: 1.9952, Val Acc: 0.5109  
Epoch 35/50 - Train Loss: 1.7561, Train Acc: 0.5338, Val Loss: 1.9777, Val Acc: 0.5147  
Epoch 36/50 - Train Loss: 1.7421, Train Acc: 0.5387, Val Loss: 2.0036, Val Acc: 0.5111  
Epoch 37/50 - Train Loss: 1.7389, Train Acc: 0.5373, Val Loss: 1.9969, Val Acc: 0.5074  
Epoch 38/50 - Train Loss: 1.7382, Train Acc: 0.5379, Val Loss: 1.9943, Val Acc: 0.5108  
Epoch 39/50 - Train Loss: 1.7302, Train Acc: 0.5412, Val Loss: 2.0225, Val Acc: 0.5039  
Epoch 40/50 - Train Loss: 1.7250, Train Acc: 0.5409, Val Loss: 2.0034, Val Acc: 0.5088  
Epoch 41/50 - Train Loss: 1.7257, Train Acc: 0.5431, Val Loss: 1.9943, Val Acc: 0.5128  
Epoch 42/50 - Train Loss: 1.7194, Train Acc: 0.5415, Val Loss: 2.0049, Val Acc: 0.5107  
Epoch 43/50 - Train Loss: 1.7132, Train Acc: 0.5443, Val Loss: 2.0169, Val Acc: 0.5077  
Epoch 44/50 - Train Loss: 1.7117, Train Acc: 0.5430, Val Loss: 2.0227, Val Acc: 0.5036  
Epoch 45/50 - Train Loss: 1.7137, Train Acc: 0.5408, Val Loss: 2.0147, Val Acc: 0.5060  
Epoch 46/50 - Train Loss: 1.7169, Train Acc: 0.5439, Val Loss: 2.0125, Val Acc: 0.5083  
Epoch 47/50 - Train Loss: 1.7077, Train Acc: 0.5419, Val Loss: 2.0302, Val Acc: 0.5057  
Epoch 48/50 - Train Loss: 1.6910, Train Acc: 0.5454, Val Loss: 2.0203, Val Acc: 0.5051  
Epoch 49/50 - Train Loss: 1.6981, Train Acc: 0.5469, Val Loss: 2.0234, Val Acc: 0.5046  
Epoch 50/50 - Train Loss: 1.6892, Train Acc: 0.5456, Val Loss: 2.0313, Val Acc: 0.5032

## Plot accuracy and loss

```
In [15]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_loss_history, label="Train Loss")
plt.plot(val_loss_history, label="Val Loss")
plt.title("Loss Curves")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(train_acc_history, label="Train Acc")
plt.plot(val_acc_history, label="Val Acc")
plt.title("Accuracy Curves")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
```

```
plt.legend()  
plt.show()
```



plot predictions

```
In [16]: model.load_state_dict(torch.load("best_model.pth"))

images, labels = next(iter(test_loader))
results = predict(model, images, device, class_names=class_names)

plt.figure(figsize=(15,12))
rows,cols = 3,3
# i had to get the transforms to reverse them to print the pictures
IMAGE_MEAN = np.array([0.485, 0.456, 0.406])
IMAGE_STD = np.array([0.229, 0.224, 0.225])

for i in range(rows*cols):
    plt.subplot(rows, cols, i + 1)

    img = images[i].cpu().numpy().transpose((1, 2, 0))
    img = img * IMAGE_STD + IMAGE_MEAN
    img = np.clip(img, 0, 1)

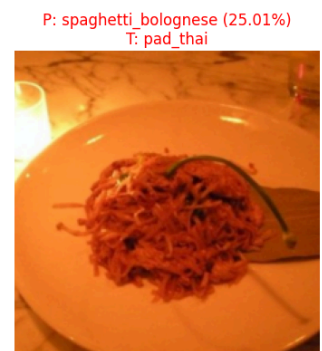
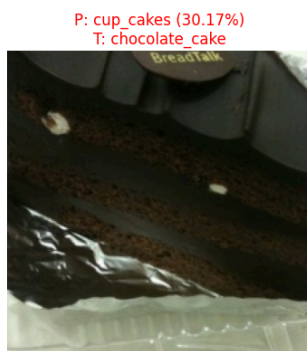
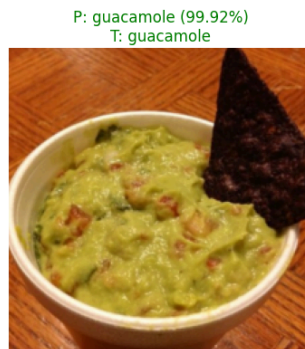
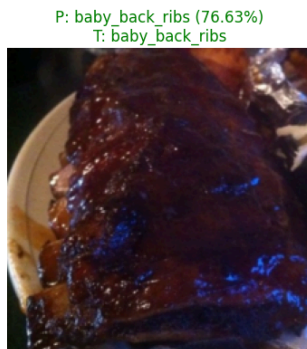
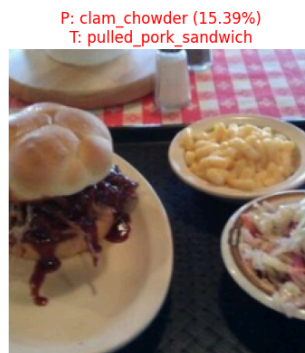
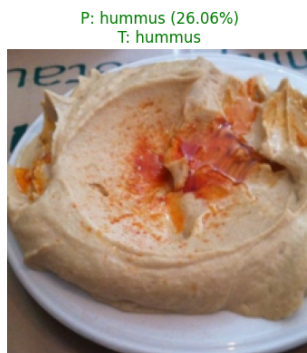
    plt.imshow(img)

    predicted_data = results[i]['topk'][0]
    pred_label = predicted_data['Label']
    p_val = predicted_data['probability']

    gt = class_names[labels[i]]

    color = "green" if pred_label == gt else "red"
    plt.title(f"P: {pred_label} ({p_val:.2%})\nT: {gt}", color=color)
    plt.axis('off')

plt.tight_layout()
plt.show()
```



## calculate TEST accuracy score

```
In [17]: test_loss, test_acc = evaluate(model, test_loader, loss_fn, device=device)
print(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc:.4f}")
```

Test Loss: 1.7088 | Test Accuracy: 0.5553

## Calculate confusion matrices precision and recall

```
In [18]: with torch.no_grad():
model.eval()
all_preds = []
all_labels = []
correct, total = 0, 0
for images, labels in test_loader:
    images, labels = images.to(device), labels.to(device)
    logits = model(images)

    preds = logits.argmax(dim=1)
    all_preds.extend(preds.cpu().tolist())
    all_labels.extend(labels.cpu().tolist())
```

```
        correct += (preds == labels).sum().item()
        total += labels.size(0)

# print(correct / total)

print(f'Number of Predictions {len(all_preds)} and, Labels {len(all_labels)}')

print(classification_report(all_labels,all_preds, target_names=class_names))

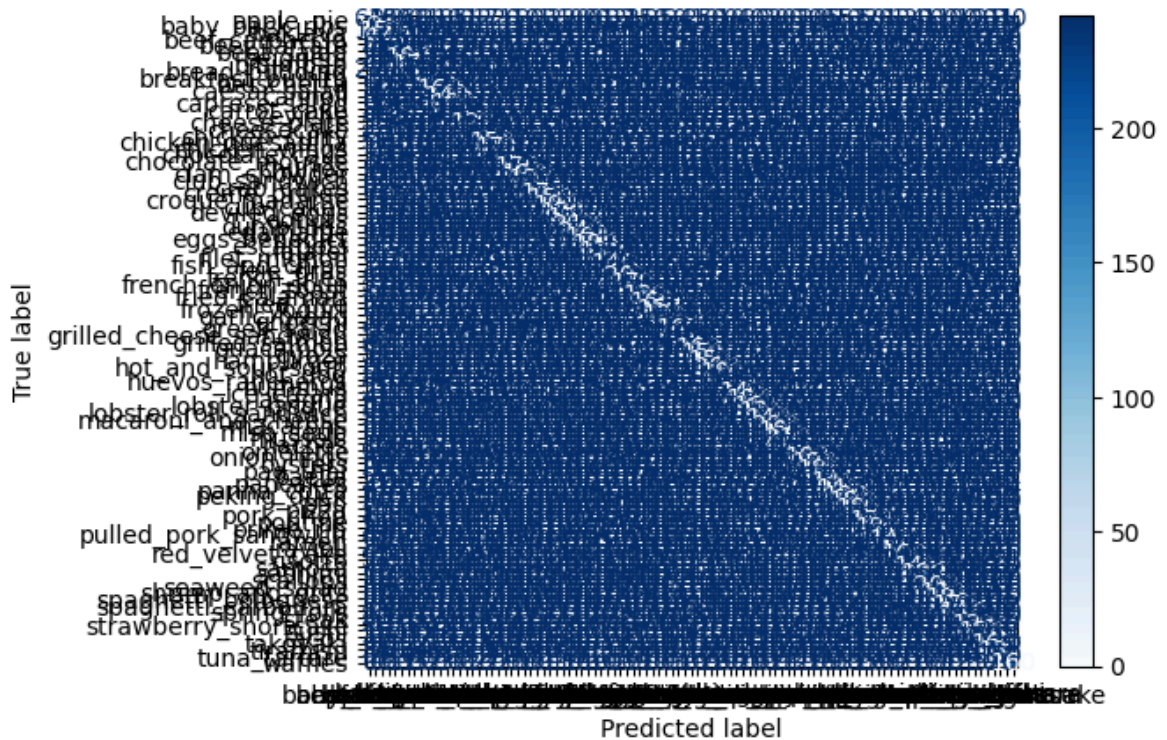
cm = confusion_matrix(all_labels,all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,display_labels = class_names)

disp.plot(cmap="Blues")
```

| Number of Predictions 25250 and, LAbels 25250 |           |        |          |         |
|-----------------------------------------------|-----------|--------|----------|---------|
|                                               | precision | recall | f1-score | support |
| apple_pie                                     | 0.28      | 0.24   | 0.26     | 250     |
| baby_back_ribs                                | 0.43      | 0.50   | 0.46     | 250     |
| baklava                                       | 0.48      | 0.61   | 0.54     | 250     |
| beef_carpaccio                                | 0.48      | 0.59   | 0.53     | 250     |
| beef_tartare                                  | 0.41      | 0.42   | 0.42     | 250     |
| beet_salad                                    | 0.47      | 0.44   | 0.46     | 250     |
| beignets                                      | 0.63      | 0.75   | 0.68     | 250     |
| bibimbap                                      | 0.77      | 0.70   | 0.74     | 250     |
| bread_pudding                                 | 0.37      | 0.29   | 0.33     | 250     |
| breakfast_burrito                             | 0.44      | 0.46   | 0.45     | 250     |
| bruschetta                                    | 0.51      | 0.35   | 0.41     | 250     |
| caesar_salad                                  | 0.47      | 0.65   | 0.54     | 250     |
| cannoli                                       | 0.61      | 0.59   | 0.60     | 250     |
| caprese_salad                                 | 0.56      | 0.50   | 0.53     | 250     |
| carrot_cake                                   | 0.60      | 0.42   | 0.49     | 250     |
| ceviche                                       | 0.38      | 0.22   | 0.28     | 250     |
| cheese_plate                                  | 0.60      | 0.49   | 0.54     | 250     |
| cheesecake                                    | 0.43      | 0.37   | 0.40     | 250     |
| chicken_curry                                 | 0.42      | 0.41   | 0.41     | 250     |
| chicken_quesadilla                            | 0.42      | 0.60   | 0.49     | 250     |
| chicken_wings                                 | 0.59      | 0.68   | 0.64     | 250     |
| chocolate_cake                                | 0.54      | 0.42   | 0.47     | 250     |
| chocolate_mousse                              | 0.50      | 0.31   | 0.39     | 250     |
| churros                                       | 0.65      | 0.70   | 0.68     | 250     |
| clam_chowder                                  | 0.63      | 0.77   | 0.69     | 250     |
| club_sandwich                                 | 0.58      | 0.55   | 0.56     | 250     |
| crab_cakes                                    | 0.38      | 0.32   | 0.35     | 250     |
| creme_brulee                                  | 0.63      | 0.69   | 0.66     | 250     |
| croque_madame                                 | 0.57      | 0.60   | 0.58     | 250     |
| cup_cakes                                     | 0.70      | 0.77   | 0.73     | 250     |
| deviled_eggs                                  | 0.63      | 0.68   | 0.66     | 250     |
| donuts                                        | 0.69      | 0.69   | 0.69     | 250     |
| dumplings                                     | 0.85      | 0.85   | 0.85     | 250     |
| edamame                                       | 0.90      | 0.97   | 0.93     | 250     |
| eggs_benedict                                 | 0.69      | 0.66   | 0.67     | 250     |
| escargots                                     | 0.64      | 0.76   | 0.69     | 250     |
| falafel                                       | 0.60      | 0.52   | 0.56     | 250     |
| filet_mignon                                  | 0.33      | 0.35   | 0.34     | 250     |
| fish_and_chips                                | 0.52      | 0.58   | 0.55     | 250     |
| foie_gras                                     | 0.34      | 0.24   | 0.28     | 250     |
| french_fries                                  | 0.59      | 0.81   | 0.68     | 250     |
| french_onion_soup                             | 0.67      | 0.69   | 0.68     | 250     |
| french_toast                                  | 0.38      | 0.46   | 0.42     | 250     |
| fried_calamari                                | 0.60      | 0.56   | 0.58     | 250     |
| fried_rice                                    | 0.70      | 0.66   | 0.68     | 250     |
| frozen_yogurt                                 | 0.92      | 0.63   | 0.75     | 250     |
| garlic_bread                                  | 0.52      | 0.50   | 0.51     | 250     |
| gnocchi                                       | 0.50      | 0.36   | 0.41     | 250     |
| greek_salad                                   | 0.42      | 0.67   | 0.51     | 250     |
| grilled_cheese_sandwich                       | 0.42      | 0.42   | 0.42     | 250     |
| grilled_salmon                                | 0.31      | 0.40   | 0.35     | 250     |
| guacamole                                     | 0.70      | 0.68   | 0.69     | 250     |
| gyoza                                         | 0.62      | 0.60   | 0.61     | 250     |
| hamburger                                     | 0.50      | 0.63   | 0.55     | 250     |
| hot_and_sour_soup                             | 0.76      | 0.83   | 0.79     | 250     |
| hot_dog                                       | 0.69      | 0.73   | 0.71     | 250     |
| huevos_rancheros                              | 0.39      | 0.18   | 0.25     | 250     |

|                       |      |      |      |       |
|-----------------------|------|------|------|-------|
| hummus                | 0.55 | 0.23 | 0.32 | 250   |
| ice_cream             | 0.59 | 0.59 | 0.59 | 250   |
| lasagna               | 0.41 | 0.50 | 0.45 | 250   |
| lobster_bisque        | 0.71 | 0.65 | 0.68 | 250   |
| lobster_roll_sandwich | 0.63 | 0.52 | 0.57 | 250   |
| macaroni_and_cheese   | 0.64 | 0.56 | 0.60 | 250   |
| macarons              | 0.77 | 0.91 | 0.83 | 250   |
| miso_soup             | 0.84 | 0.73 | 0.78 | 250   |
| mussels               | 0.59 | 0.84 | 0.70 | 250   |
| nachos                | 0.52 | 0.31 | 0.39 | 250   |
| omelette              | 0.36 | 0.31 | 0.33 | 250   |
| onion_rings           | 0.77 | 0.73 | 0.75 | 250   |
| oysters               | 0.81 | 0.79 | 0.80 | 250   |
| pad_thai              | 0.62 | 0.74 | 0.68 | 250   |
| paella                | 0.63 | 0.43 | 0.51 | 250   |
| pancakes              | 0.55 | 0.56 | 0.56 | 250   |
| panna_cotta           | 0.44 | 0.56 | 0.49 | 250   |
| peking_duck           | 0.51 | 0.56 | 0.53 | 250   |
| pho                   | 0.76 | 0.75 | 0.75 | 250   |
| pizza                 | 0.64 | 0.74 | 0.69 | 250   |
| pork_chop             | 0.26 | 0.28 | 0.27 | 250   |
| poutine               | 0.65 | 0.69 | 0.67 | 250   |
| prime_rib             | 0.53 | 0.51 | 0.52 | 250   |
| pulled_pork_sandwich  | 0.49 | 0.37 | 0.42 | 250   |
| ramen                 | 0.53 | 0.59 | 0.56 | 250   |
| ravioli               | 0.32 | 0.31 | 0.32 | 250   |
| red_velvet_cake       | 0.64 | 0.67 | 0.66 | 250   |
| risotto               | 0.50 | 0.38 | 0.43 | 250   |
| samosa                | 0.51 | 0.58 | 0.54 | 250   |
| sashimi               | 0.83 | 0.73 | 0.78 | 250   |
| scallops              | 0.45 | 0.38 | 0.41 | 250   |
| seaweed_salad         | 0.64 | 0.81 | 0.72 | 250   |
| shrimp_and_grits      | 0.38 | 0.52 | 0.44 | 250   |
| spaghetti_bolognese   | 0.62 | 0.74 | 0.68 | 250   |
| spaghetti_carbonara   | 0.89 | 0.71 | 0.79 | 250   |
| spring_rolls          | 0.64 | 0.58 | 0.61 | 250   |
| steak                 | 0.32 | 0.22 | 0.26 | 250   |
| strawberry_shortcake  | 0.55 | 0.51 | 0.53 | 250   |
| sushi                 | 0.45 | 0.62 | 0.52 | 250   |
| tacos                 | 0.32 | 0.32 | 0.32 | 250   |
| takoyaki              | 0.53 | 0.65 | 0.59 | 250   |
| tiramisu              | 0.38 | 0.52 | 0.44 | 250   |
| tuna_tartare          | 0.42 | 0.23 | 0.30 | 250   |
| waffles               | 0.62 | 0.64 | 0.63 | 250   |
| accuracy              |      |      | 0.56 | 25250 |
| macro avg             | 0.56 | 0.56 | 0.55 | 25250 |
| weighted avg          | 0.56 | 0.56 | 0.55 | 25250 |

Out[18]: <sklearn.metrics.\_plot.confusion\_matrix.ConfusionMatrixDisplay at 0x7fae68cce660>



I cant make sense out of this confusion matrix. should I only focus on the errors?  
or just plot the top errors? 101 classes make no sense. or I can plot with 20 classes?

```
In [19]: # Zeroing out the correct guesses to see only the errors
np.fill_diagonal(cm, 0)
top_errors = np.unravel_index(np.argsort(cm, axis=None)[-5:], cm.shape)

print("\nTop 5 Most Frequent Confusions:")
for i in range(4, -1, -1): # Printing the highest errors first
    true_idx, pred_idx = top_errors[0][i], top_errors[1][i]
    count = cm[true_idx, pred_idx]
    print(f"True: {class_names[true_idx]} mistook for {class_names[pred_idx]} {count} times")
```

Top 5 Most Frequent Confusions:

True: pulled\_pork\_sandwich mistook for hamburger (51 times)

True: steak mistook for filet\_mignon (33 times)

True: pork\_chop mistook for filet\_mignon (33 times)

True: chocolate\_cake mistook for tiramisu (31 times)

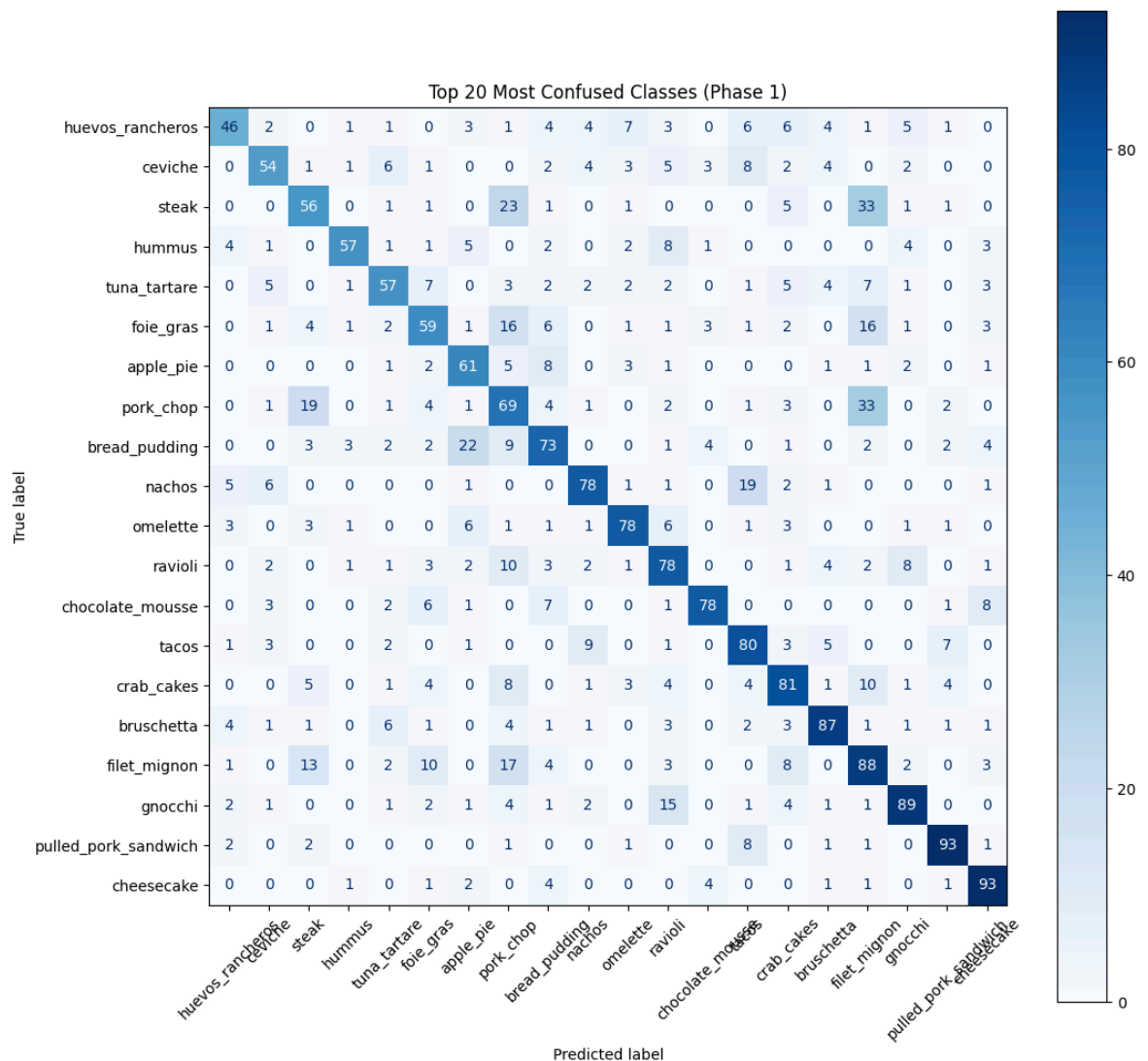
True: ceviche mistook for greek\_salad (30 times)

```
In [ ]: cm = confusion_matrix(all_labels, all_preds)

troublesome_indices = np.argsort(cm.diagonal())[:20]
cm_small = cm[troublesome_indices][:, troublesome_indices]
subset_names = [class_names[i] for i in troublesome_indices]

fig, ax = plt.subplots(figsize=(12, 12))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_small, display_labels=subset_names)
disp.plot(cmap="Blues", ax=ax, xticks_rotation=45, values_format='d')

plt.title("Top 20 Most Confused Classes (Phase 1)")
plt.show()
```



Phase 2: Freeze layer 4 - Fine tuning

from the freezed cnn unfreeze the layer4.

Train for 50 epochs

```
In [21]: model_params = {
    "number_of_output_classes":101,
    "freeze_backbone": True,
    "dense_units":[256],
    "dropout_probabilities":0.3
}
model = ResNet18Transfer(**model_params)

model.load_state_dict(torch.load("best_model.pth"))

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
model.fine_tune(unfreeze=True)
```

```
trainable_params = [p for p in model.parameters() if p.requires_grad]
print(f"Number of trainable parameters groups: {len(trainable_params)}")

optimizer_ft = optim.Adam([p for p in model.parameters() if p.requires_grad])
loss_fn = nn.CrossEntropyLoss()

epochs_ft = 50
best_val_acc_ft = 0.0
ft_train_loss_hist, ft_val_loss_hist = [], []
ft_train_acc_hist, ft_val_acc_hist = [], []

for epoch in range(epochs_ft):
    train_loss, train_acc = train_one_epoch(model, train_loader, optimizer_ft, device=device)
    val_loss, val_acc = evaluate(model, val_loader, loss_fn, device=device)

    ft_train_loss_hist.append(train_loss)
    ft_val_loss_hist.append(val_loss)
    ft_train_acc_hist.append(train_acc)
    ft_val_acc_hist.append(val_acc)

    print(f"FT Epoch {epoch+1}/{epochs_ft} | Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f}")

    if val_acc > best_val_acc_ft:
        best_val_acc_ft = val_acc
        torch.save(model.state_dict(), "best_model_finetuned.pth")
```

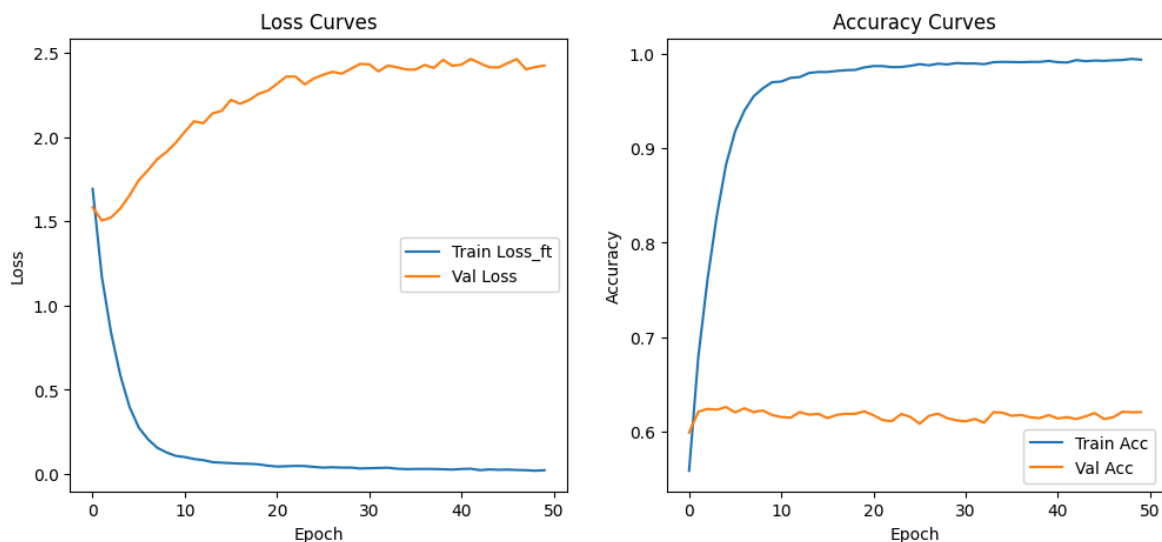
Number of trainable parameters groups: 19

|                |                   |                 |
|----------------|-------------------|-----------------|
| FT Epoch 1/50  | Train Acc: 0.5588 | Val Acc: 0.5991 |
| FT Epoch 2/50  | Train Acc: 0.6795 | Val Acc: 0.6211 |
| FT Epoch 3/50  | Train Acc: 0.7614 | Val Acc: 0.6242 |
| FT Epoch 4/50  | Train Acc: 0.8284 | Val Acc: 0.6233 |
| FT Epoch 5/50  | Train Acc: 0.8824 | Val Acc: 0.6262 |
| FT Epoch 6/50  | Train Acc: 0.9182 | Val Acc: 0.6205 |
| FT Epoch 7/50  | Train Acc: 0.9399 | Val Acc: 0.6249 |
| FT Epoch 8/50  | Train Acc: 0.9550 | Val Acc: 0.6208 |
| FT Epoch 9/50  | Train Acc: 0.9635 | Val Acc: 0.6225 |
| FT Epoch 10/50 | Train Acc: 0.9699 | Val Acc: 0.6177 |
| FT Epoch 11/50 | Train Acc: 0.9706 | Val Acc: 0.6156 |
| FT Epoch 12/50 | Train Acc: 0.9745 | Val Acc: 0.6148 |
| FT Epoch 13/50 | Train Acc: 0.9753 | Val Acc: 0.6209 |
| FT Epoch 14/50 | Train Acc: 0.9797 | Val Acc: 0.6180 |
| FT Epoch 15/50 | Train Acc: 0.9807 | Val Acc: 0.6190 |
| FT Epoch 16/50 | Train Acc: 0.9807 | Val Acc: 0.6145 |
| FT Epoch 17/50 | Train Acc: 0.9818 | Val Acc: 0.6178 |
| FT Epoch 18/50 | Train Acc: 0.9826 | Val Acc: 0.6191 |
| FT Epoch 19/50 | Train Acc: 0.9829 | Val Acc: 0.6189 |
| FT Epoch 20/50 | Train Acc: 0.9856 | Val Acc: 0.6217 |
| FT Epoch 21/50 | Train Acc: 0.9870 | Val Acc: 0.6174 |
| FT Epoch 22/50 | Train Acc: 0.9869 | Val Acc: 0.6123 |
| FT Epoch 23/50 | Train Acc: 0.9859 | Val Acc: 0.6113 |
| FT Epoch 24/50 | Train Acc: 0.9860 | Val Acc: 0.6189 |
| FT Epoch 25/50 | Train Acc: 0.9872 | Val Acc: 0.6157 |
| FT Epoch 26/50 | Train Acc: 0.9890 | Val Acc: 0.6084 |
| FT Epoch 27/50 | Train Acc: 0.9878 | Val Acc: 0.6168 |
| FT Epoch 28/50 | Train Acc: 0.9895 | Val Acc: 0.6192 |
| FT Epoch 29/50 | Train Acc: 0.9887 | Val Acc: 0.6143 |
| FT Epoch 30/50 | Train Acc: 0.9902 | Val Acc: 0.6121 |
| FT Epoch 31/50 | Train Acc: 0.9897 | Val Acc: 0.6110 |
| FT Epoch 32/50 | Train Acc: 0.9897 | Val Acc: 0.6135 |
| FT Epoch 33/50 | Train Acc: 0.9890 | Val Acc: 0.6096 |
| FT Epoch 34/50 | Train Acc: 0.9911 | Val Acc: 0.6208 |
| FT Epoch 35/50 | Train Acc: 0.9914 | Val Acc: 0.6201 |
| FT Epoch 36/50 | Train Acc: 0.9913 | Val Acc: 0.6168 |
| FT Epoch 37/50 | Train Acc: 0.9911 | Val Acc: 0.6178 |
| FT Epoch 38/50 | Train Acc: 0.9914 | Val Acc: 0.6153 |
| FT Epoch 39/50 | Train Acc: 0.9914 | Val Acc: 0.6144 |
| FT Epoch 40/50 | Train Acc: 0.9924 | Val Acc: 0.6176 |
| FT Epoch 41/50 | Train Acc: 0.9911 | Val Acc: 0.6141 |
| FT Epoch 42/50 | Train Acc: 0.9907 | Val Acc: 0.6155 |
| FT Epoch 43/50 | Train Acc: 0.9933 | Val Acc: 0.6134 |
| FT Epoch 44/50 | Train Acc: 0.9921 | Val Acc: 0.6164 |
| FT Epoch 45/50 | Train Acc: 0.9928 | Val Acc: 0.6197 |
| FT Epoch 46/50 | Train Acc: 0.9925 | Val Acc: 0.6134 |
| FT Epoch 47/50 | Train Acc: 0.9931 | Val Acc: 0.6154 |
| FT Epoch 48/50 | Train Acc: 0.9934 | Val Acc: 0.6213 |
| FT Epoch 49/50 | Train Acc: 0.9946 | Val Acc: 0.6206 |
| FT Epoch 50/50 | Train Acc: 0.9938 | Val Acc: 0.6208 |

## Plot curves

```
In [22]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(ft_train_loss_hist, label="Train Loss_ft")
plt.plot(ft_val_loss_hist, label="Val Loss")
```

```
plt.title("Loss Curves")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(ft_train_acc_hist, label="Train Acc")
plt.plot(ft_val_acc_hist, label="Val Acc")
plt.title("Accuracy Curves")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



## visualize prediction

```
In [23]: model.load_state_dict(torch.load("best_model_finetuned.pth"))

images, labels = next(iter(test_loader))
results = predict(model, images, device, class_names=class_names)

plt.figure(figsize=(15,12))
rows,cols = 3,3
# i had to get the transforms to reverse them to print the pictures
IMAGE_MEAN = np.array([0.485, 0.456, 0.406])
IMAGE_STD = np.array([0.229, 0.224, 0.225])

for i in range(rows*cols):
    plt.subplot(rows, cols, i + 1)

    img = images[i].cpu().numpy().transpose((1, 2, 0))
    img = img * IMAGE_STD + IMAGE_MEAN
    img = np.clip(img, 0, 1)

    plt.imshow(img)

    predicted_data = results[i]['topk'][0]
    pred_label = predicted_data['Label']
    p_val = predicted_data['probability']
```

```

gt = class_names[labels[i]]

color = "green" if pred_label == gt else "red"
plt.title(f"P: {pred_label} ({p_val:.2%})\nT: {gt}", color=color)
plt.axis('off')

plt.tight_layout()
plt.show()

```

P: fried\_rice (23.61%)  
T: nachos



P: chocolate\_mousse (88.07%)  
T: tiramisu



P: ice\_cream (39.28%)  
T: ice\_cream



P: ramen (99.77%)  
T: ramen



P: tiramisu (22.57%)  
T: tiramisu



P: ramen (61.40%)  
T: pad\_thai



P: carrot\_cake (84.82%)  
T: carrot\_cake



P: sashimi (99.57%)  
T: sashimi



P: macarons (32.80%)  
T: garlic\_bread



## Calculate test accuracy score

```

In [24]: test_loss, test_acc = evaluate(model, test_loader, loss_fn, device=device)
print(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc:.4f}")

```

Test Loss: 1.3132 | Test Accuracy: 0.6768

## calculate confusion matrix precision and recall

```

In [25]: with torch.no_grad():
model.eval()
all_preds = []

```

```
all_labels = []
correct, total = 0,0
for images, labels in test_loader:
    images, labels = images.to(device), labels.to(device)
    logits = model(images)

    preds = logits.argmax(dim=1)
    all_preds.extend(preds.cpu().tolist())
    all_labels.extend(labels.cpu().tolist())

    correct += (preds == labels).sum().item()
    total += labels.size(0)

# print(correct / total)

print(f'Number of Predictions {len(all_preds)} and, Labels {len(all_labels)}')

print(classification_report(all_labels,all_preds, target_names=class_names))

cm = confusion_matrix(all_labels,all_preds)

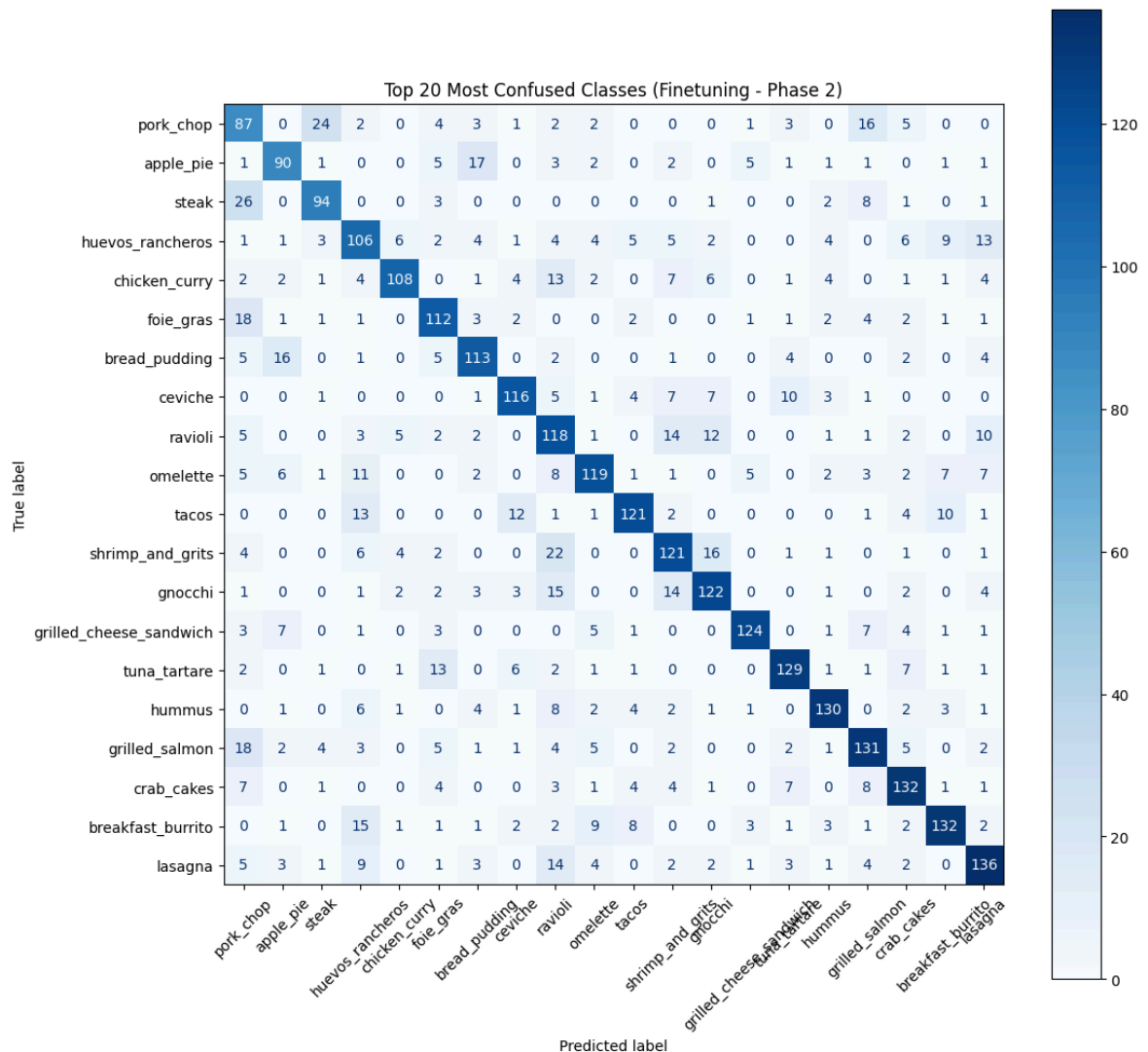
troublesome_indices = np.argsort(cm.diagonal()[::-1])[20:]
cm_small = cm[troublesome_indices][:, troublesome_indices]
subset_names = [class_names[i] for i in troublesome_indices]

fig, ax = plt.subplots(figsize=(12, 12))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_small, display_labels=subset_names)
disp.plot(cmap="Blues", ax=ax, xticks_rotation=45, values_format='d')

plt.title("Top 20 Most Confused Classes (Finetuning - Phase 2)")
plt.show()
```

| Number of Predictions 25250 and, Labels 25250 |           |        |          |         |
|-----------------------------------------------|-----------|--------|----------|---------|
|                                               | precision | recall | f1-score | support |
| apple_pie                                     | 0.42      | 0.36   | 0.39     | 250     |
| baby_back_ribs                                | 0.71      | 0.65   | 0.68     | 250     |
| baklava                                       | 0.67      | 0.77   | 0.72     | 250     |
| beef_carpaccio                                | 0.69      | 0.72   | 0.70     | 250     |
| beef_tartare                                  | 0.61      | 0.62   | 0.62     | 250     |
| beet_salad                                    | 0.52      | 0.63   | 0.57     | 250     |
| beignets                                      | 0.79      | 0.78   | 0.79     | 250     |
| bibimbap                                      | 0.82      | 0.82   | 0.82     | 250     |
| bread_pudding                                 | 0.45      | 0.45   | 0.45     | 250     |
| breakfast_burrito                             | 0.62      | 0.53   | 0.57     | 250     |
| bruschetta                                    | 0.47      | 0.62   | 0.53     | 250     |
| caesar_salad                                  | 0.76      | 0.72   | 0.74     | 250     |
| cannoli                                       | 0.74      | 0.72   | 0.73     | 250     |
| caprese_salad                                 | 0.60      | 0.73   | 0.66     | 250     |
| carrot_cake                                   | 0.64      | 0.66   | 0.65     | 250     |
| ceviche                                       | 0.50      | 0.46   | 0.48     | 250     |
| cheese_plate                                  | 0.71      | 0.68   | 0.70     | 250     |
| cheesecake                                    | 0.55      | 0.60   | 0.57     | 250     |
| chicken_curry                                 | 0.64      | 0.43   | 0.52     | 250     |
| chicken_quesadilla                            | 0.77      | 0.67   | 0.71     | 250     |
| chicken_wings                                 | 0.68      | 0.79   | 0.73     | 250     |
| chocolate_cake                                | 0.64      | 0.64   | 0.64     | 250     |
| chocolate_mousse                              | 0.49      | 0.55   | 0.52     | 250     |
| churros                                       | 0.85      | 0.79   | 0.82     | 250     |
| clam_chowder                                  | 0.81      | 0.79   | 0.80     | 250     |
| club_sandwich                                 | 0.79      | 0.69   | 0.74     | 250     |
| crab_cakes                                    | 0.46      | 0.53   | 0.49     | 250     |
| creme_brulee                                  | 0.82      | 0.79   | 0.81     | 250     |
| croque_madame                                 | 0.75      | 0.75   | 0.75     | 250     |
| cup_cakes                                     | 0.77      | 0.84   | 0.80     | 250     |
| deviled_eggs                                  | 0.77      | 0.82   | 0.79     | 250     |
| donuts                                        | 0.73      | 0.69   | 0.71     | 250     |
| dumplings                                     | 0.88      | 0.87   | 0.87     | 250     |
| edamame                                       | 0.97      | 0.98   | 0.97     | 250     |
| eggs_benedict                                 | 0.84      | 0.71   | 0.77     | 250     |
| escargots                                     | 0.76      | 0.82   | 0.79     | 250     |
| falafel                                       | 0.67      | 0.58   | 0.62     | 250     |
| filet_mignon                                  | 0.40      | 0.62   | 0.48     | 250     |
| fish_and_chips                                | 0.84      | 0.60   | 0.70     | 250     |
| foie_gras                                     | 0.36      | 0.45   | 0.40     | 250     |
| french_fries                                  | 0.77      | 0.86   | 0.82     | 250     |
| french_onion_soup                             | 0.73      | 0.78   | 0.75     | 250     |
| french_toast                                  | 0.61      | 0.58   | 0.60     | 250     |
| fried_calamari                                | 0.72      | 0.68   | 0.70     | 250     |
| fried_rice                                    | 0.81      | 0.73   | 0.77     | 250     |
| frozen_yogurt                                 | 0.80      | 0.82   | 0.81     | 250     |
| garlic_bread                                  | 0.63      | 0.58   | 0.61     | 250     |
| gnocchi                                       | 0.60      | 0.49   | 0.54     | 250     |
| greek_salad                                   | 0.61      | 0.71   | 0.66     | 250     |
| grilled_cheese_sandwich                       | 0.55      | 0.50   | 0.52     | 250     |
| grilled_salmon                                | 0.52      | 0.52   | 0.52     | 250     |
| guacamole                                     | 0.84      | 0.75   | 0.79     | 250     |
| gyoza                                         | 0.76      | 0.73   | 0.74     | 250     |
| hamburger                                     | 0.74      | 0.62   | 0.68     | 250     |
| hot_and_sour_soup                             | 0.84      | 0.86   | 0.85     | 250     |
| hot_dog                                       | 0.76      | 0.74   | 0.75     | 250     |
| huevos_rancheros                              | 0.40      | 0.42   | 0.41     | 250     |

|                       |      |      |      |       |
|-----------------------|------|------|------|-------|
| hummus                | 0.60 | 0.52 | 0.56 | 250   |
| ice_cream             | 0.65 | 0.64 | 0.65 | 250   |
| lasagna               | 0.58 | 0.54 | 0.56 | 250   |
| lobster_bisque        | 0.85 | 0.68 | 0.75 | 250   |
| lobster_roll_sandwich | 0.79 | 0.71 | 0.75 | 250   |
| macaroni_and_cheese   | 0.66 | 0.65 | 0.66 | 250   |
| macarons              | 0.94 | 0.89 | 0.92 | 250   |
| miso_soup             | 0.88 | 0.84 | 0.86 | 250   |
| mussels               | 0.85 | 0.83 | 0.84 | 250   |
| nachos                | 0.58 | 0.64 | 0.61 | 250   |
| omelette              | 0.50 | 0.48 | 0.49 | 250   |
| onion_rings           | 0.84 | 0.83 | 0.84 | 250   |
| oysters               | 0.90 | 0.86 | 0.88 | 250   |
| pad_thai              | 0.82 | 0.76 | 0.79 | 250   |
| paella                | 0.69 | 0.70 | 0.69 | 250   |
| pancakes              | 0.70 | 0.74 | 0.72 | 250   |
| panna_cotta           | 0.64 | 0.69 | 0.66 | 250   |
| peking_duck           | 0.76 | 0.65 | 0.70 | 250   |
| pho                   | 0.85 | 0.82 | 0.84 | 250   |
| pizza                 | 0.78 | 0.75 | 0.77 | 250   |
| pork_chop             | 0.32 | 0.35 | 0.33 | 250   |
| poutine               | 0.80 | 0.71 | 0.75 | 250   |
| prime_rib             | 0.77 | 0.72 | 0.75 | 250   |
| pulled_pork_sandwich  | 0.71 | 0.62 | 0.66 | 250   |
| ramen                 | 0.67 | 0.73 | 0.70 | 250   |
| ravioli               | 0.40 | 0.47 | 0.43 | 250   |
| red_velvet_cake       | 0.78 | 0.82 | 0.80 | 250   |
| risotto               | 0.53 | 0.66 | 0.59 | 250   |
| samosa                | 0.79 | 0.65 | 0.71 | 250   |
| sashimi               | 0.83 | 0.81 | 0.82 | 250   |
| scallops              | 0.42 | 0.62 | 0.50 | 250   |
| seaweed_salad         | 0.82 | 0.85 | 0.84 | 250   |
| shrimp_and_grits      | 0.52 | 0.48 | 0.50 | 250   |
| spaghetti_bolognese   | 0.78 | 0.87 | 0.82 | 250   |
| spaghetti_carbonara   | 0.84 | 0.88 | 0.86 | 250   |
| spring_rolls          | 0.78 | 0.70 | 0.74 | 250   |
| steak                 | 0.41 | 0.38 | 0.39 | 250   |
| strawberry_shortcake  | 0.70 | 0.68 | 0.69 | 250   |
| sushi                 | 0.71 | 0.71 | 0.71 | 250   |
| tacos                 | 0.49 | 0.48 | 0.49 | 250   |
| takoyaki              | 0.87 | 0.72 | 0.79 | 250   |
| tiramisu              | 0.65 | 0.61 | 0.63 | 250   |
| tuna_tartare          | 0.43 | 0.52 | 0.47 | 250   |
| waffles               | 0.74 | 0.78 | 0.76 | 250   |
| accuracy              |      |      | 0.68 | 25250 |
| macro avg             | 0.69 | 0.68 | 0.68 | 25250 |
| weighted avg          | 0.69 | 0.68 | 0.68 | 25250 |



## Question 2 What did you learn? What is the difference between transfer learning and fine tuning?

So in transfer learning, I am taking an already trained model and only training a new classifier to classify foods from my food 101 dataset. Here the transfer learning had an accuracy of 55% on the test set. But in the case of fine-tuning, I had to unfreeze the layer 4. to make sure that I do not run into overfitting on the train-set beacuse of the layer 4, I had to set a weight decay rate of 1e-4. Even after that, the train accuracy was very high and validation accuranty stayed around 62%. But good news is the model performed with a 67.68% accuracy on the test set.

Overall I think the previous weight from transfer learning were pretty good to be able to get an accuracy of 55%. On the other hand, fine tuning is hard because it is very easy to overfit the model on the train set. But finetunning improved the model performance as it could train a bit more and specialize on the food classification task.